H2S-NN: Heterogeneous Heterophilic Sheaf Neural Network

Anonymous Author(s)

Abstract

Learning on heterogeneous heterophilic graphs requires addressing two properties. First, heterogeneity means a graph contains multiple node and relation types with distinct semantic roles. Second, heterophily means that connected nodes may differ in labels or features, making similarity-based neighbor aggregation unreliable. Another important aspect is the use of meta-path schemas, which expose typed multi-hop semantics. Many existing meta-path methods represent concrete path instances through endpoint-level views, discarding both the internal path structure and the higher-order information it carries. Therefore, we introduce the Heterogeneous Heterophilic Sheaf Neural Network (H2S-NN), which retains the observed graph structure and represents meta-path instances as path-supported 2-cells, thereby preserving graph heterogeneity. Across the evaluated node-classification, graph-classification, and link-prediction benchmarks, H2S-NN achieved the strongest overall performance among the compared methods, supporting its effectiveness for heterogeneous heterophilic graph learning.

CCS Concepts

• **Computing methodologies** → **Neural networks**.

Keywords

heterogeneous graph neural networks, heterophily, sheaf neural networks, meta-path instances, diffusion

ACM Reference Format:

Anonymous Author(s). 2026. H2S-NN: Heterogeneous Heterophilic Sheaf Neural Network. In *Proceedings of ACM Conference (Conference acronym 'XX)*. ACM, New York, NY, USA, 14 pages. https://doi.org/XXXXXXX.XXXXXXX

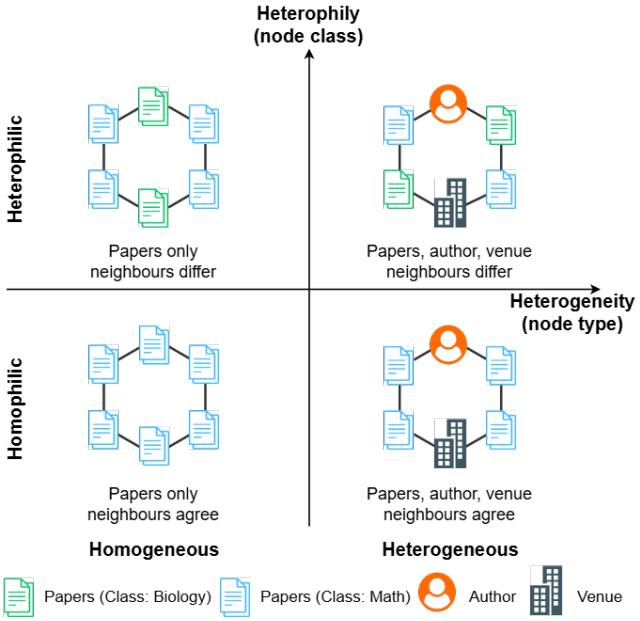


Figure 1: An example of a heterogeneous heterophilic graph. The graph contains multiple node and relation types, and adjacent nodes may have different labels or attributes.

1 Introduction

Heterogeneous graphs are a common representation for modeling complex relational structures, such as citation networks [21] or social interactions [31]. Unlike homogeneous graphs, heterogeneous graphs contain multiple types of nodes and relations with distinct semantic roles [32]. Moreover, many graphs are also heterophilic, meaning that adjacent nodes may have different labels or attributes, making simple similarity-based neighbor aggregation unreliable and prone to oversmoothing. A graph can be both heterogeneous and heterophilic, resulting in a complex learning task called heterogeneous heterophilic graph learning, as shown in Figure 1. Furthermore, modeling graphs under the simplifying assumptions of homogeneity or homophily can obscure important structural and relational patterns. These simplifying assumptions can produce an incomplete representation of the original graph and consequently degrade predictive performance [20].

Many existing methods address either heterogeneous graphs without explicitly considering heterophily [14, 17, 24, 38, 39, 41] or heterophilic graphs without modeling heterogeneous node and relation types [2, 10, 29, 40]. Recent methods have begun to address heterogeneous heterophilic graphs through meta-path-schema-based latent subgraphs [5, 19, 22, 30, 37], graph rewiring [11, 34], graph transformers [20], and sheaf-based mappings [3]. Another important concept is the use of meta-path schemas. These schemas can express information that is not contained in either node endpoint alone [33]. For instance, an APA (author-paper-author) meta-path schema describes a co-authorship relation in citation networks, as shown in Figure 2. However, existing methods either ignore meta-path schemas completely or use them as semantic views such as filtered edge sets or aggregation channels, rather than retaining each concrete path instance as part of the propagation structure. For example, an APA meta-path schema can connect two authors through a shared paper, but after it is converted into an author-author view, the shared paper and the two authorship edges no longer directly shape the propagation operator. This node-endpoint-only representation is restrictive under heterophily because whether information should be reinforced or contrasted may depend on the relations and

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, City, Country

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2026/MM
https://doi.org/XXXXXXX.XXXXXXX

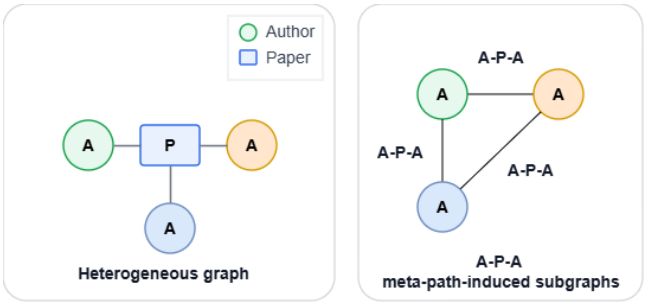


Figure 2: Illustration of meta-path-schema-induced graph transformation. Under the APA meta-path schema, authors that share a paper in the original heterogeneous graph are connected in an author-level subgraph.

intermediate nodes along the path. Collapsing the path can therefore discard local relational structure and higher-order information carried by the concrete path instance.

To address this gap, we introduce the **Heterogeneous Heterophilic Sheaf Neural Network (H2S-NN)**, which represents concrete meta-path instances as path-supported 2-cells and incorporates them into a sheaf-Hodge propagation operator. Compatible node-to-edge and edge-to-face maps preserve the ordered relational context of each path while allowing relation-specific transformations of node and edge states. H2S-NN also supports automatic task-aligned selection of meta-paths. Our main contributions are summarized as follows:

(1) We propose the Heterogeneous Heterophilic Sheaf Neural Network (H2S-NN), which retains concrete path instances as path-supported 2-cells in an augmented path-supported 2-dimensional cellular complex, thereby preserving both the original graph structure and local relational structure under heterophily.
(2) We develop Adaptive Meta-Path Schema-to-Cell Searching and Selection (AMPSS), a component of H2S-NN that selects relevant meta-path schemas and identifies their task-relevant concrete instances.
(3) We develop Sheaf Covariance Pooling (SheafCovPool), a graph-level readout component of H2S-NN that produces permutation- and gauge-invariant graph representations for graph-level tasks.

As a result, H2S-NN provides an effective and generalizable model for heterogeneous heterophilic graphs.

2 Preliminaries

2.1 Heterogeneous Graphs

A heterogeneous graph is a tuple $G = (V, E, \phi, \psi)$, where V and E are the node and edge sets, $\phi : V \to \mathcal{T}$ assigns each node a type, and $\psi : E \to \mathcal{R}$ assigns each edge a relation type. The type sets are $\mathcal{T} = \{\phi(v) : v \in V\}$ and $\mathcal{R} = \{\psi(e) : e \in E\}$. The graph is heterogeneous when it contains more than one node type or more than one relation type, i.e., $|\mathcal{T}| > 1$ or $|\mathcal{R}| > 1$; it reduces to a homogeneous graph when $|\mathcal{T}| = |\mathcal{R}| = 1$ [32].

2.2 Meta-Path Schemas and Path Instances

A meta-path schema of length ℓ is a typed relation sequence

$$\pi : t_0 \xrightarrow{r_1} t_1 \xrightarrow{r_2} \cdots \xrightarrow{r_\ell} t_\ell, \qquad t_i \in \mathcal{T},\ r_i \in \mathcal{R}, \tag{1}$$

which specifies a composite relation between its endpoint types [33]. For each relation r, let $A_r \in \{0,1\}^{|V|\times|V|}$ be its adjacency matrix, with $(A_r)_{uv} = 1$ when a relation of type r connects u to v. The schema induces the walk-count matrix

$$A_\pi = A_{r_1} A_{r_2} \cdots A_{r_\ell}, \tag{2}$$

where $(A_\pi)_{uv}$ counts the concrete walks from u to v that follow π. For example, as shown in Figure 2, the APA schema has $A_{\mathrm{APA}} = A_{\mathrm{AP}}A_{\mathrm{PA}}$ and connects authors that share a paper. A concrete path instance is an ordered sequence $f = (v_0, e_1, v_1, \ldots, e_\ell, v_\ell)$ satisfying $\phi(v_i) = t_i$ and $\psi(e_i) = r_i$. Thus, π describes a type-level pattern, whereas f identifies one occurrence of that pattern in the observed graph.

2.3 Homophily and Heterophily

Homophily refers to the prevalence of edges between nodes that share the same label. For a graph $G' = (V', E')$ whose endpoint labels are comparable, its edge homophily is

$$H_{\mathrm{edge}}(G') = \frac{1}{|E'|} \sum_{(u,v)\in E'} \mathbf{1}[y_u = y_v], \tag{3}$$

where y_u is the label of node u and $\mathbf{1}[\cdot]$ is the indicator function [43]. Heterophily is the complementary tendency for adjacent nodes to have different labels. In a heterogeneous graph, labels are generally comparable only after a meta-path induces an endpoint graph whose nodes share the target type.

We therefore characterize heterogeneous node-classification graphs using meta-path-based label heterophily (MLH) [20]. Let $\mathcal{P}_2(t_{\mathrm{tgt}})$ be the set of valid length-two meta-path schemas that start and end at the target node type t_{tgt}, and let G'_π denote the endpoint graph induced by $\pi \in \mathcal{P}_2(t_{\mathrm{tgt}})$. For each G'_π, define edge heterophily as

$$H_{\mathrm{het}}(G'_\pi) = 1 - H_{\mathrm{edge}}(G'_\pi). \tag{4}$$

We use the mean aggregation function over these induced graphs:

$$\mathrm{MLH}(G; t_{\mathrm{tgt}}) = \frac{1}{|\mathcal{P}_2(t_{\mathrm{tgt}})|} \sum_{\pi\in\mathcal{P}_2(t_{\mathrm{tgt}})} H_{\mathrm{het}}(G'_\pi). \tag{5}$$

MLH ranges from 0 to 1, with lower values indicating homophily and higher values indicating heterophily. For multilabel targets, two nodes are treated as having the same label only when their label vectors are identical.

2.4 Cellular Sheaves and Sheaf Neural Networks

In sheaf theory, a graph is viewed as a one-dimensional cellular complex, with nodes as 0-cells and edges as 1-cells. More general cellular complexes may include higher-dimensional cells, such as 2-cells bounded by edges, but most sheaf neural networks on graphs operate on the node-edge complex. We write $c \trianglelefteq \tau$ when a lower-dimensional cell c is incident to a higher-dimensional cell τ.

A cellular sheaf $\mathcal{F}$ assigns each cell c a vector space, or *stalk*, $\mathcal{F}(c)$, and each incident pair $c \trianglelefteq \tau$ a linear restriction map $F_{c\trianglelefteq\tau}$:

$\mathcal{F}(c) \to \mathcal{F}(\tau)$. On a graph $G = (V, E)$, node and edge assignments form the cochain spaces

$$C^0(G, \mathcal{F}) = \bigoplus_{v \in V} \mathcal{F}(v), \qquad C^1(G, \mathcal{F}) = \bigoplus_{e \in E} \mathcal{F}(e). \tag{6}$$

For an oriented edge $e = (u \to v)$, the first coboundary operator $\delta_0 : C^0(G, \mathcal{F}) \to C^1(G, \mathcal{F})$ compares endpoint signals after transport into the edge stalk:

$$(\delta_0 x)_e = F_{v \trianglelefteq e} x_v - F_{u \trianglelefteq e} x_u. \tag{7}$$

The corresponding sheaf Laplacian is $L_0 = \delta_0^\top \delta_0$, with energy $x^\top L_0 x = \|\delta_0 x\|_2^2$. When all stalks are one-dimensional and all restriction maps are identities, L_0 recovers the ordinary graph Laplacian.

Sheaf neural networks can address oversmoothing [4] and heterophily by replacing direct feature averaging with diffusion through local stalks and learned restriction maps [2, 12], leading to numerous recent extensions and applications of sheaf neural networks on graphs and in other domains [1, 9, 16, 27–29]. HetSheaf further adapts sheaf maps to heterogeneous graphs [3]. However, these models define sheaf structure primarily over observed node-edge incidences. In contrast, H2S-NN retains selected concrete path instances as path-supported 2-cells in an augmented path-supported 2-dimensional cellular complex and constructs compatible edge-to-face maps, enabling diffusion over an explicit edge cochain.

3 Methodology

3.1 Problem Formulation

We consider supervised prediction on a directed heterogeneous graph in which neighboring nodes may have different labels or features. The graph is formulated as $G = (V, E, \phi, \psi, X^{\text{raw}})$, where $\phi : V \to \mathcal{T}$ assigns node types, $\psi : E \to \mathcal{R}$ assigns relation types, and $X^{\text{raw}} = \{x_v^{\text{raw}}\}_{v \in V}$ contains type-dependent node features with $x_v^{\text{raw}} \in \mathbb{R}^{d_{\phi(v)}}$. The model receives a set of meta-path schemas S. A schema $\pi = (t_0, r_1, t_1, \ldots, r_\ell, t_\ell)$ specifies a valid typed relation sequence, while $f = (v_0, e_1, v_1, \ldots, e_\ell, v_\ell)$ denotes one concrete path instance satisfying $\phi(v_i) = t_i$ and $\psi(e_i) = r_i$. Let $\mathcal{I}_\pi$ be the set of observed instances of schema π, and let $\mathcal{I}_S = \bigcup_{\pi \in S} \mathcal{I}_\pi$.

The learning problem is to construct a predictor

$$\hat{y}_o = g_\theta(o; G, \mathcal{I}_S),$$

for a task-specific supervised object o. For node classification, o is a target node; for link prediction, o is a candidate typed node pair; and for graph classification, o is an entire graph. Given labeled training objects $\mathcal{S}_{\text{train}}$, the parameters are learned by minimizing

$$\min_\theta \frac{1}{|\mathcal{S}_{\text{train}}|} \sum_{o \in \mathcal{S}_{\text{train}}} \mathcal{L}(g_\theta(o; G, \mathcal{I}_S), y_o)\,.$$

$\mathcal{L}$ is the cross-entropy loss for single-label node and graph classification, and binary cross-entropy for multilabel node classification and link prediction. Thus, the central challenge is to learn type-aware and relation-aware propagation operators that can use selected concrete path instances as path-supported 2-cells on heterogeneous heterophilic graphs.

3.2 Overview

We introduce H2S-NN, a Heterogeneous Heterophilic Sheaf Neural Network, for supervised learning on heterogeneous heterophilic graphs. As shown in Figure 3, H2S-NN consists of four stages. First, *Input graph preprocessing* prepares typed graph features and relations. Second, AMPSS generates candidate meta-path schemas using bounded depth-first search, selects a task-relevant schema set (Stage I), and learns schema-conditioned weights for its retained concrete instances (Stage II). Third, the *Sheaf-Hodge Encoding* stage incorporates the selected concrete path instances to learn node-to-edge and path-supported edge-to-face geometry. Finally, the *Sheaf-Hodge Diffusion* stage evolves the node and edge cochains to obtain their final representations, with the proposed SheafCov-Pool provided as the graph-level readout. Together, these stages allow H2S-NN to retain the local relational structure of selected concrete path instances while learning from heterogeneous graphs under heterophily.

3.3 Input Graph Preprocessing: Initialization of Node Stalks and Sheaf Restrictions

H2S-NN first maps type-dependent node features into a shared d-dimensional local feature space (stalk space). This makes representations from different node types compatible with relation-specific restrictions, which transform the endpoint representations into a common edge-stalk space before their disagreement is computed. For each node v, we compute

$$h_v^{(0)} = \text{Dropout}\big[\text{GELU}\big(\text{LN}\left(W_{\phi(v)} x_v^{\text{raw}} + b_{\phi(v)}\right) + \nu_{\phi(v)}\big)\big] \in \mathbb{R}^d, \tag{8}$$

where W_t and b_t are type-specific parameters and ν_t is a learned type embedding. When raw attributes are unavailable, we use a trainable node embedding $\varepsilon_v \in \mathbb{R}^d$ directly as the initial representation $h_v^{(0)}$. We initialize the node cochain as $x^{(0)} = h^{(0)}$ and associate each relation r with an embedding $\zeta_r \in \mathbb{R}^{d_r}$.

A restriction map must decide whether two endpoints should agree or disagree, hence we condition it on a one-round relation-wise summary of each endpoint's neighborhood rather than on the endpoint alone:

$$\widetilde{h}_v^{(0)} = \text{LN}\left(h_v^{(0)} + \sum_{r \in \mathcal{R}} W_r^{\text{ctx}} \frac{1}{\max(|\mathcal{N}_r(v)|, 1)} \sum_{u \in \mathcal{N}_r(v)} h_u^{(0)}\right) \in \mathbb{R}^d, \tag{9}$$

where $\mathcal{N}_r(v)$ contains the nodes joined to v by a relation of type r, and $W_r^{\text{ctx}} \in \mathbb{R}^{d \times d}$ is a per-relation parameter.

For an oriented typed edge $e = (u \xrightarrow{r} v)$, two relation-conditioned networks infer the endpoint restrictions $F_{u \trianglelefteq e}, F_{v \trianglelefteq e} \in \mathbb{R}^{d \times d}$ from $\widetilde{h}_u^{(0)}$, $\widetilde{h}_v^{(0)}$, and ζ_r. The source and target restrictions use the same parameterization:

$$\begin{aligned} F_{u \trianglelefteq e} &= \lambda_{u,e} Q_{u,e}, & Q_{u,e}^\top Q_{u,e} &= I_d, & \lambda_{u,e} &> 0, \\ F_{v \trianglelefteq e} &= \lambda_{v,e} Q_{v,e}, & Q_{v,e}^\top Q_{v,e} &= I_d, & \lambda_{v,e} &> 0. \end{aligned} \tag{10}$$

Here, the orthogonal factor aligns endpoint coordinates, while the positive scale modulates the magnitude of the relation-aware disagreement computed by δ_0, so that endpoints are compared after alignment rather than in their raw feature spaces.

Because $Q_{v,e}$ is orthogonal, the node block of L_0 is a scalar multiple of the identity,

$$D_v = \sum_{e \ni v} F_{v \trianglelefteq e}^\top F_{v \trianglelefteq e} = d_v I_d, \qquad d_v = \sum_{e \ni v} \lambda_{v,e}^2, \tag{11}$$

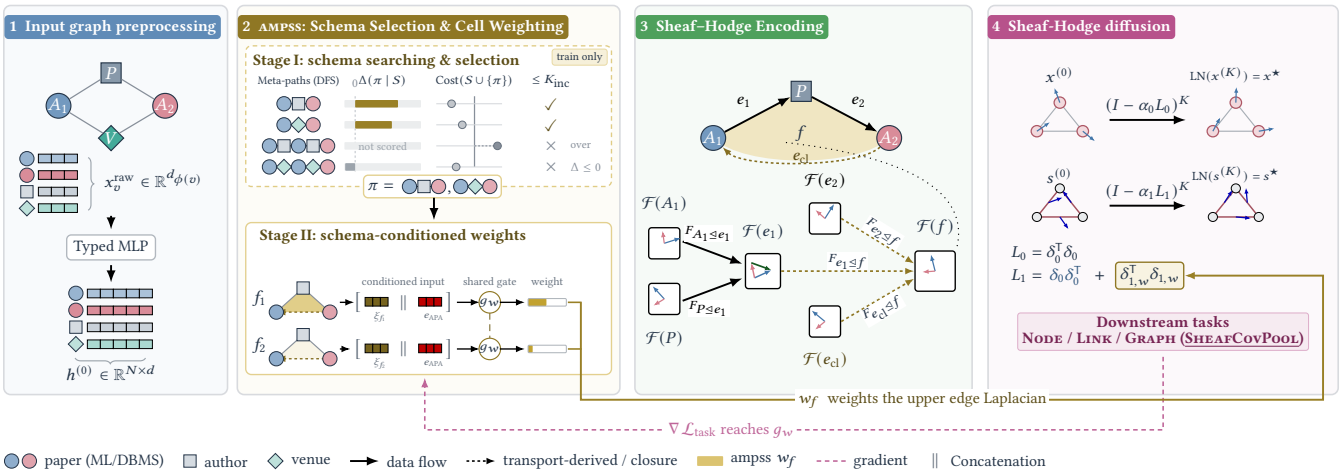


Figure 3: Overview of H2S-NN on a citation graph. (1) Type-specific encoders map heterogeneous node features to a common d-dimensional stalk representation. (2) AMPSS enumerates meta-path candidates by depth-first search (DFS) and fits an auxiliary linear selection model, fitted only on training data, to select meta-path schemas in Stage I. In Stage II, it learns schema-conditioned weights for the retained concrete path instances. (3) Each selected simple path instance is completed by a synthetic closure 1-cell to form a path-supported 2-cell. Learned node-to-edge maps on observed edges define path transports, which construct the closure restrictions and compatible edge-to-face maps in the corresponding face-stalk frame. (4) Node and edge cochains undergo K parallel Hodge-diffusion steps before task-specific prediction. The learned instance weights modulate the face-induced edge interactions, allowing the supervised task loss to update the Stage-II instance weights jointly with the predictor.

where the sum runs over the incidences at v in the extended edge set E_{ext} of Section 3.5.1. Writing $\bar{d}_v = d_v + 1$ for the degree guarded against nodes of zero incidence, we obtain the normalized sheaf Laplacian $D^{-1/2} L_0 D^{-1/2}$ [13] by absorbing $\bar{d}_v^{-1/2}$ into $\lambda_{v,e}$. This scaling keeps every restriction map inside the scaled-orthogonal family of Equation (10) and bounds the spectrum of L_0 by 2, so the explicit steps in Equation (25) remain stable across the searched step sizes. We initialize the edge cochain independently of the node cochain:

$$s_e^{(0)} = \text{MLP}_{\text{init}}\left(h_u^{(0)} \,\|\, h_v^{(0)} \,\|\, \zeta_r \,\|\, (\delta_0 h^{(0)})_e\right). \tag{12}$$

Thus, $s^{(0)}$ need not lie in $\operatorname{im} \delta_0$, the subspace of edge cochains that some node assignment can produce, allowing the subsequent node-mediated and face-induced edge interactions to act on edge-level information that is not induced solely by node representations. After the selected path instances introduce closure 1-cells, the same coboundary and initialization rules are evaluated over the extended edge set, with a dedicated learned token ζ_{cl} replacing ζ_r in Equation (12).

3.4 AMPSS: Adaptive Meta-Path Schema-to-Cell Searching and Selection

AMPSS constructs a task-specific set of meta-path instances in two stages. Stage I generates candidate meta-path schemas through bounded depth-first search and selects a task-aligned subset of meta-path schemas from the training data. Stage II assigns schema-conditioned weights to the retained path instances, controlling their contribution to the face-induced edge operator. As a result, we obtain a task-adaptive higher-order edge operator that prioritizes relevant typed interactions while differentially weighting their concrete path instances.

3.4.1 Stage I: Task-Aligned Schema Searching and Selection. Stage I identifies a compact set of task-relevant meta-path schemas. We construct a type graph from the relation types available during Stage I and use bounded depth-first search to generate a candidate pool Π of schemas $\pi = (t_0, r_1, t_1, \ldots, r_\ell, t_\ell)$, with lengths between $\ell_{\min}$ and $\ell_{\max}$. Candidates with no feasible simple path instances are discarded.

For a trial schema set $S \subseteq \Pi$, deterministic instance and incidence quotas define the retained instances $\mathcal{I}_\pi(S)$, the post-quota retained set of schema π under S. Each schema's retained instances are summarized into features for an auxiliary linear selection model and also determine the structural cost. Stage I greedily selects schemas subject to a schema limit K_S, an instance budget K_{inst}, and an incidence budget K_{inc}.

An edge-face incidence is one occurrence of a boundary edge in a path-supported 2-cell. The structural cost is the number of retained edge-face incidences:

$$\text{Cost}(S) = \sum_{\pi \in S} \sum_{f \in \mathcal{I}_\pi(S)} (\ell_f + 1), \tag{13}$$

where each instance contributes its ℓ_f path edges and one closure edge, and $\text{Inst}(S) = \sum_{\pi \in S} |\mathcal{I}_\pi(S)|$ counts the retained instances. Thus, the incidence budget directly bounds the support of the resulting face-induced edge operator.

The auxiliary linear selection model is fitted on an inner-training partition and evaluated on a disjoint held-out selection partition.

The gain from adding a candidate schema π is

$$\Delta(\pi \mid S) = \mathcal{L}_{\text{sel}}(S) - \mathcal{L}_{\text{sel}}(S \cup \{\pi\}), \qquad \pi \in \mathcal{A}(S),$$
$$\mathcal{A}(S) = \left\{ \pi \in \Pi \setminus S \;\middle|\; \begin{array}{r} |S \cup \{\pi\}| \le K_S, \\ \text{Inst}(S \cup \{\pi\}) \le K_{\text{inst}}, \\ \text{Cost}(S \cup \{\pi\}) \le K_{\text{inc}} \end{array} \right\}. \tag{14}$$

Starting from $S = \varnothing$, Stage I adds the feasible schema that yields the largest positive reduction in held-out selection loss. The selection model is refitted for each trial set using labels from the outer training split only. For link prediction, inner-selection positive links and their reverse counterparts are excluded from the graph used for selection. After selection, the retained instances are constructed on the full outer-training graph. Because schema selection is discrete, gradients from the H2S-NN task loss do not update Stage I. Thus, Stage I provides a task-aligned schema set for Stage II.

3.4.2 Stage II: Schema-Conditioned Instance Weighting. For the schema pool selected in Stage I, Stage II assigns a positive weight to each retained concrete path instance. Instances sharing the same meta-path schema can encode different task-relevant information. Let $\xi_f \in \mathbb{R}^{d_\xi}$ denote the ordered-path latent that the face encoder of Equation (18) produces before mapping it to Θ_f, and let $\zeta_{\pi(f)}$ denote a learned embedding of the schema of f. We define

$$w_f(\mathrm{t}) = (1 - \beta(\mathrm{t})) + 2\beta(\mathrm{t})\ \text{sigmoid}\big(g_w(\xi_f \,\|\, \zeta_{\pi(f)})\big), \tag{15}$$

where t is the training epoch and $\beta(\mathrm{t}) \in [0, 1]$ is a warm-up schedule that gradually changes the weights from uniform to learned values. The schema embedding enables the weighting function to distinguish instances from different schemas, such as PAP and PSP, whereas ξ_f distinguishes instances within the same schema. The weights are learned through the supervised task loss.

Let $\mathcal{W}(\mathrm{t}) = \bigoplus_{f \in \mathcal{I}_{\text{ret}}} w_f(\mathrm{t}) I_d$ be the block-diagonal operator on $C^2(\mathcal{F})$ that scales each instance's face stalk by $w_f(\mathrm{t})$. Stage II weights the face-induced component of the edge operator as $\delta_{1,w}^{\mathsf{T}} \delta_{1,w} = \delta_1^{\mathsf{T}} \mathcal{W}(\mathrm{t}) \delta_1$, where $\delta_{1,w}$ is defined in Equation (21) and δ_1 denotes the unweighted second coboundary, that is, Equation (21) with $w_f \equiv 1$. Because $w_f(\mathrm{t}) > 0$, this component remains positive semidefinite. The weights therefore modulate only the contribution of retained path-supported 2-cells. Closure edges remain part of the extended edge domain and continue to contribute to δ_0, L_0, the node-mediated component of the edge operator, the weighted face boundary in $\delta_{1,w}$, and the edge readout.

Thus, Stage II provides task-driven, instance-level modulation of the selected path-supported 2-cells while preserving the extended edge structure.

3.5 Sheaf-Hodge Encoding

Diffusion requires nodes, edges, and selected path instances to already carry compatible sheaf geometry. We introduce a Sheaf-Hodge encoding to address this requirement. It augments the observed graph with path-supported 2-cells and defines compatible node-to-edge and edge-to-face restriction maps, preserving the ordered structure of each selected path while enabling compatible diffusion on the node and edge cochains through shared restriction geometry and retained higher-order supports.

3.5.1 Augmented Path-Supported 2-Cell Construction. The Sheaf-Hodge encoder augments the observed graph with retained concrete path instances. In AMPSS mode, Stage I selects the corresponding meta-path schemas and determines the retained instances through its quotas. Rather than replacing an instance with an endpoint-only relation, H2S-NN retains the ordered path

$$f = (v_0, e_1, v_1, \ldots, e_\ell, v_\ell).$$

Because this path is open, we augment it with a synthetic closure 1-cell, $e_{\text{cl}} = (v_\ell \to v_0)$, thereby forming a path-supported 2-cell whose closed boundary preserves the ordered path edges:

$$\partial f = (e_1, \ldots, e_\ell, e_{\text{cl}}). \tag{16}$$

The closure 1-cell is computationally active in the extended edge domain, with its restriction maps determined by path transport rather than learned independently.

Let E_{cl} contain one closure 1-cell for each retained instance, and let $E_{\text{ext}} = E \sqcup E_{\text{cl}}$. The retained instance set is

$$\mathcal{I}_{\text{ret}} = \bigcup_{\pi \in S} \mathcal{I}_\pi(S),$$

where S denotes the selected schema set. With d-dimensional node, edge, and face stalks, the cochain spaces are

$$C^0(\mathcal{F}) = \bigoplus_{v \in V} \mathcal{F}(v),\ C^1(\mathcal{F}) = \bigoplus_{e \in E_{\text{ext}}} \mathcal{F}(e),\ C^2(\mathcal{F}) = \bigoplus_{f \in \mathcal{I}_{\text{ret}}} \mathcal{F}(f). \tag{17}$$

After the closure restrictions are defined by the path transport described below, the first coboundary in Equation (7) is evaluated over E_{ext}. Hence, closure 1-cells are included in the extended edge domain and participate in the initial edge cochain and in both L_0 and L_1.

3.5.2 Path-Supported Edge-to-Face Maps. The edge-face construction attaches a path-supported 2-cell to each selected concrete path instance. For a selected PAP instance, for example, the two paper-author edges remain the boundary edges of one path-supported 2-cell. The intermediate author remains part of the operator through the ordered path boundary, while the synthetic closure edge adds a path-transport-constrained coupling between the endpoints. This allows the face-induced edge interactions to couple disagreements along the whole PAP pattern.

For a general instance $f = (v_0, e_1, v_1, \ldots, e_\ell, v_\ell)$, H2S-NN first predicts an unconstrained map $\Theta_f \in \mathbb{R}^{d \times d}$ from the ordered path. Each face group contains path instances of a single schema, so all instances in the group have the same length ℓ and relation sequence $(r_1, \ldots, r_\ell)$. The default face encoder concatenates the node states in path order together with the schema's relation embeddings:

$$\Theta_f = \text{MLP}_\ell\Big(h_{v_0}^{(0)} \,\|\, \cdots \,\|\, h_{v_\ell}^{(0)} \,\|\, \zeta_{r_1} \,\|\, \cdots \,\|\, \zeta_{r_\ell}\Big) \in \mathbb{R}^{d \times d}. \tag{18}$$

Here, MLP_ℓ is a path-length-specific MLP that maps the ordered path representation to the entries of Θ_f through an intermediate ordered-path latent ξ_f, the instance descriptor used in Equation (15). We construct the edge-to-face maps by transporting Θ_f along the ordered path through the node-to-edge restriction maps. This construction ensures that the face-induced component measures edge-level inconsistency around a selected path-supported 2-cell, rather than creating an artificial face signal from an edge cochain induced

solely by node representations. Starting from the base vertex v_0, the transport is

$$T_{v_0} = \Theta_f, \qquad T_{v_i} = T_{v_{i-1}} F_{u \trianglelefteq e_i}^{-1} F_{v \trianglelefteq e_i}, \qquad F_{e_i \trianglelefteq f} = T_{v_{i-1}} F_{u \trianglelefteq e_i}^{-1}, \tag{19}$$

where $u = v_{i-1}$ and $v = v_i$. Unrolling the recursion gives $T_{v_\ell} = \Theta_f H_f$, where $H_f = M_1 M_2 \cdots M_\ell$ and $M_i = F_{u \trianglelefteq e_i}^{-1} F_{v \trianglelefteq e_i}$, the factors being ordered along the path. We call H_f the *holonomy* of f: it is the transport accumulated by traversing the path from v_0 to v_ℓ. These inverses exist for every value of the learned parameters because Equation (10) makes each node-to-edge map a positive scale times an orthogonal matrix, giving the closed form $F^{-1} = F^\top/\lambda^2$.

The closure edge uses the same path transport:

$$F_{v_0 \trianglelefteq e_{\mathrm{cl}}} = I_d, \qquad F_{v_\ell \trianglelefteq e_{\mathrm{cl}}} = H_f, \qquad F_{e_{\mathrm{cl}} \trianglelefteq f} = \Theta_f. \tag{20}$$

Since the source-side closure map is the holonomy H_f, the closure edge introduces no additional learned restriction map.

Path edge e_i is stored as $v_{i-1} \to v_i$ and the closure 1-cell as $v_\ell \to v_0$, so path-walking ∂f from v_0 traverses every boundary edge forward. All incidence signs are +1, and the weighted second coboundary is

$$(\delta_{1,w} s)_f = \sqrt{\frac{w_f}{\bar{d}_{v_0}}} \sum_{e \in \partial f} F_{e \trianglelefteq f} s_e, \tag{21}$$

where v_0 is the base vertex of f and $\bar{d}_{v_0}$ is the guarded degree of Equation (11). The transport in Equation (19) carries the base vertex's normalization into every boundary map, so dividing the face row by $\sqrt{\bar{d}_{v_0}}$ removes it again and leaves a face-induced operator that does not depend on the degree of v_0.

Writing $B_f s = \sum_{e \in \partial f} F_{e \trianglelefteq f} s_e$, the corresponding face-induced contribution is

$$\delta_{1,w}^\top \delta_{1,w} = \sum_{f \in \mathcal{I}_{\mathrm{ret}}} \frac{w_f}{\bar{d}_{v_0(f)}} B_f^\top B_f. \tag{22}$$

Because the edge-to-face maps are obtained by transport rather than emitted independently, $\delta_{1,w}\delta_0 = 0$ holds identically for every value of the learned parameters, so $C^0 \to C^1 \to C^2$ is a cochain complex.

3.6 Sheaf-Hodge Diffusion

After the restriction maps have been constructed, H2S-NN diffuses node and edge cochains using one node operator and one edge operator:

$$L_0 x = \delta_0^\top \delta_0 x, \tag{23}$$

$$L_1 s = \underbrace{\delta_0 \delta_0^\top s}_{\text{node-mediated component}} + \underbrace{\delta_{1,w}^\top \delta_{1,w} s}_{\text{face-induced component}}. \tag{24}$$

The node operator L_0 compares transported endpoint signals. The edge diffusion operator L_1 contains a node-mediated component and a face-induced component. The node-mediated component of L_1 relates edge signals through shared incident nodes, whereas the face-induced component couples boundary edges within selected path-supported 2-cells. In Hodge-theoretic terminology, these are the lower and upper components of the edge operator, which represent node-mediated and face-induced edge interactions, respectively. Because each closure 1-cell is part of the extended edge set, it participates in both components without requiring a separate node-side correction.

Starting from $x^{(0)} = h^{(0)}$ and the independently initialized edge cochain $s^{(0)}$, K explicit Euler steps give

$$x^{(k+1)} = x^{(k)} - \alpha_0 L_0 x^{(k)}, \qquad s^{(k+1)} = s^{(k)} - \alpha_1 L_1 s^{(k)}, \qquad k = 0, \ldots, K-1. \tag{25}$$

The positive step sizes may be learned. Node and edge cochains are updated by separate L_0 and L_1 operators. Although updated node and edge states are not exchanged between diffusion steps, both operators share the learned restriction geometry and initial node representations. Layer normalization then gives the terminal states $x^\star$ and $s^\star$, which are passed to the task-specific readout. The Hodge operators are applied matrix-free: δ_0, $\delta_0^\top$, $\delta_{1,w}$, and $\delta_{1,w}^\top$ act on node, edge, and face states through their local restriction maps, so no global operator is materialized. $\delta_{1,w}$ uses the complete boundary cycle, consisting of the ℓ path edges and the closure 1-cell. The exact transport argument assumes simple paths, so instances that revisit a node, including coincident endpoints, are excluded.

Subsequently, the shared backbone produces final node states $x^\star$ and edge states $s^\star$. For node classification, the target-node state is combined with pooled incident-edge information. For link prediction, endpoint representations are formed from node and incident-edge states before applying a pair decoder. For graph classification, SheafCovPool aggregates node and edge representations into a graph-level vector, as described next.

3.7 SheafCovPool

To construct a graph-level representation from states expressed in different local coordinates, SheafCovPool builds a coordinate-consistent graph representation by combining complementary node and edge information (Figure 4). It first transports each incident edge state into its endpoint node space using the transpose of the corresponding restriction map:

$$\widetilde{s_v^\star} = \sum_{e:\mathrm{src}(e)=v} F_{v \trianglelefteq e}^\top s_e^\star + \sum_{e:\mathrm{tgt}(e)=v} F_{v \trianglelefteq e}^\top s_e^\star. \tag{26}$$

Here, $F_{v \trianglelefteq e}$ denotes the restriction map from node v into the stalk of an incident edge e, and the two sums enumerate the edges for which v is the source and the target, respectively. Both endpoints contribute positively because this quantity summarizes incident context rather than the orientation-sensitive coboundary adjoint $\delta_0^\top s^\star$. The sums range over the observed and closure edges, so both types of 1-cell contribute.

Let $C_0(\mathrm{g}) = V(\mathrm{g})$ and $C_1(\mathrm{g}) = E_{\mathrm{ext}}(\mathrm{g})$ denote the node set and the extended edge set of a graph g, that is, the index sets of the cochain spaces $C^0(\mathcal{F})$ and $C^1(\mathcal{F})$ in Equation (17); accordingly, $C_1(\mathrm{g})$ contains both the observed and the closure 1-cells. The readout then forms three complementary views at each node $v \in C_0(\mathrm{g})$

and each edge $e \in C_1(\mathrm{g})$:

$$\begin{aligned} Z_v^{(0)} &= \left[x_v^\star \quad h_v^{(0)} \quad \widetilde{s}_v^\star\right] \in \mathbb{R}^{d\times 3}, \\ Z_e^{(1)} &= \left[(\delta_0 x^\star)_e \quad (\delta_0 h^{(0)})_e \quad s_e^\star\right] \in \mathbb{R}^{d\times 3}, \\ \mathcal{G}_i^{(q)} &= (Z_i^{(q)})^\top Z_i^{(q)} \in \mathbb{R}^{3\times 3}, \qquad q \in \{0,1\}, \end{aligned} \tag{27}$$

where the cell index i ranges over $C_q(\mathrm{g})$, so that $Z_i^{(0)} = Z_v^{(0)}$ for a node and $Z_i^{(1)} = Z_e^{(1)}$ for an edge.

Each Gram matrix records the energies and pairwise alignments of the three views while removing their common orthogonal orientation. For an ordered view stack, equality of Gram matrices characterizes equality up to an orthogonal change of basis.

To see why the transported node view has this property, let $Q_v, R_e \in O(d)$ be independent changes of basis in the node and edge stalks. Under the consistent transformation

$$\begin{aligned} F'_{v\trianglelefteq e} &= R_e F_{v\trianglelefteq e} Q_v^\top, \qquad s_e^{\star\prime} = R_e s_e^\star, \\ (F'_{v\trianglelefteq e})^\top s_e^{\star\prime} &= Q_v F_{v\trianglelefteq e}^\top s_e^\star. \end{aligned} \tag{28}$$

Hence, $Z_v^{(0)}$ transforms by Q_v, while $Z_e^{(1)}$ transforms by R_e, and both Gram descriptors remain unchanged. This argument requires orthogonal basis changes; the learned restrictions themselves may be scaled orthogonal maps.

Nodes and edges serve different roles, so separate networks project their Gram matrices. Within graph g, attention over these invariant descriptors yields first- and second-order summaries:

$$\begin{aligned} \chi_i^{(q)} &= \rho_q(\mathrm{vec}\,\mathcal{G}_i^{(q)}) \in \mathbb{R}^{d_p}, \\ a_i^{(q)} &= \frac{\exp(g_q(\chi_i^{(q)}))}{\sum_{j\in C_q(\mathrm{g})} \exp(g_q(\chi_j^{(q)}))}, \\ \mu_q(\mathrm{g}) &= \sum_{i\in C_q(\mathrm{g})} a_i^{(q)} \chi_i^{(q)}, \\ \Sigma_q(\mathrm{g}) &= \sum_{i\in C_q(\mathrm{g})} a_i^{(q)} (\chi_i^{(q)} - \mu_q(\mathrm{g}))(\chi_i^{(q)} - \mu_q(\mathrm{g}))^\top. \end{aligned} \tag{29}$$

Attention is normalized within each graph, so the weights $a_i^{(q)}$ sum to one over $C_q(\mathrm{g})$. The mean describes the evidence emphasized by attention, whereas the covariance records how that evidence varies across the graph. The final graph representation is

$$\gamma_{\mathrm{g}} = \left[\mu_0(\mathrm{g}) \,\|\, \mathrm{triu}(\Sigma_0(\mathrm{g})) \,\|\, \mu_1(\mathrm{g}) \,\|\, \mathrm{triu}(\Sigma_1(\mathrm{g}))\right]. \tag{30}$$

The raw Gram matrix has fixed order three, corresponding to the three semantic views in each stack. Readout capacity is instead controlled by the post-Gram projection width d_p. The two means and two upper-triangular covariance vectors yield $\gamma_{\mathrm{g}} \in \mathbb{R}^{d_p^2+3d_p}$. Since all learned scores operate on invariant Gram matrices and all aggregations are permutation-invariant, γ_{g} is invariant to node and edge permutations. It is also invariant to independent orthogonal basis changes (i.e., gauge invariant) in the node and edge stalks, provided that the states and restriction maps are transformed consistently.

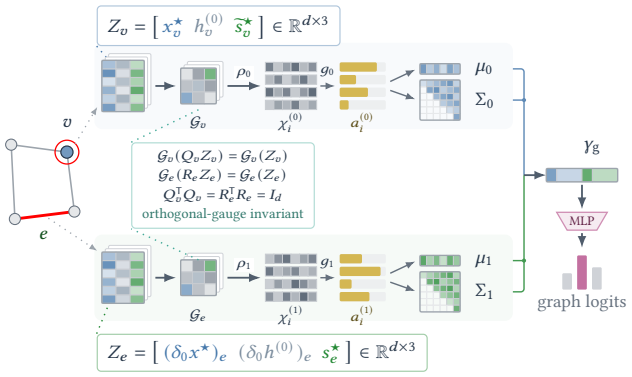


Figure 4: SheafCovPool graph readout. Incident edge states are first expressed in their endpoint node stalks. Node and edge view stacks then become gauge-invariant Gram matrices $\mathcal{G}_i^{(q)}$; attention-weighted means and covariances form the graph representation γ_{g}.

4 Experiments

4.1 Experiment Settings

To assess the generalizability of H2S-NN across heterogeneous graph learning settings, we evaluated it on three tasks: node classification, link prediction, and graph classification.

Datasets. For node classification, we used three heterogeneous graph benchmarks from HGB [23]: DBLP, ACM, and IMDB. We also computed meta-path-based label heterophily (MLH) for each dataset. For graph classification, we selected four datasets from TUDataset [26]: two biological datasets, PROTEINS and ENZYMES, and two chemical compound datasets, MUTAG and NCI1. To construct heterogeneous graphs, we interpreted the node and edge labels provided by TUDataset as node and relation types, respectively. For link prediction, we used four HGB benchmarks [23]: YouTube, a social network; LastFM, a user-item interaction network; Amazon, an e-commerce product network; and PubMed, a biological knowledge graph.

Baselines. For node classification, we compared H2S-NN with baselines from four model categories. The homogeneous homophilic baselines comprised GCN [18] and GAT [36], while the homogeneous heterophilic baselines included ChebNetII [15], NSD [2], and CSNN [29]. For heterogeneous graphs, we considered the homophilic models HAN [38], SeHGNN [39], HINormer [24], and PSHGCN [14], as well as the heterophilic models H2G-former [20], Hetero^{2}Net [19], H2SGNN [22], HetSheaf [3], and LatGRL [30]. For graph classification and link prediction, we used a subset of these baselines while retaining representatives from all four categories. Specifically, we included GCN [18] and GAT [36] as homogeneous homophilic models; NSD [2] and CSNN [29] as homogeneous heterophilic models; HAN [38] and PSHGCN [14] as heterogeneous homophilic models; and H2G-former [20], Hetero^{2}Net [19], H2SGNN [22], and HetSheaf [3] as heterogeneous heterophilic models.

Evaluation settings. For node classification, we followed the HGB benchmark protocol [23], using transductive training/validation/test

splits of 24%/6%/70% and reporting Macro-F1 and Micro-F1. For graph classification, following prior work [6, 26, 42], we performed 10-fold cross-validation over five random seeds and reported the mean and standard deviation of Macro-F1 and Micro-F1. For link prediction, we followed the HGB protocol [23], using transductive splits of 81%/9%/10% and reporting AUPR and ROC-AUC. Negative edges were sampled from the 2-hop neighborhood at a 1:1 ratio to positive edges. All models were trained with early stopping using a patience of 100 epochs. We grouped the baselines according to graph type and homophily assumption as Hom./hom., Hom./het., Het./hom., and Het./het., where the first term indicates whether the graph is modeled as homogeneous or heterogeneous and the second indicates whether the method is designed for homophilic or heterophilic settings. Results were averaged over five random seeds and reported with standard deviations. The code is available at https://anonymous.4open.science/r/H2S-NN/.

4.2 Results

For node classification, the results in Table 1 show that H2S-NN achieved the best performance on all three benchmark datasets, including both low-heterophily datasets (DBLP and ACM) and the high-heterophily dataset (IMDB), outperforming all compared baselines. These results indicate that H2S-NN can effectively capture complex heterogeneous graph structures and relationships, leading to improved node classification performance.

Moreover, for graph classification, Table 2 shows that H2S-NN achieved the highest mean Macro-F1 and Micro-F1 on PROTEINS, ENZYMES, and NCI1. On MUTAG, H2S-NN achieved the second-best performance. These results demonstrate that H2S-NN, combined with SheafCovPool, can effectively perform graph-level prediction and remains competitive across the evaluated graph classification benchmarks.

The link-prediction results in Table 3 show that H2S-NN achieved the best performance on all four datasets under both reported metrics. This advantage is consistent across multi-node-type graphs (LastFM and PubMed) and single-node-type, multi-relation graphs (Amazon and YouTube). These results indicate that H2S-NN supports link prediction on both multi-node-type graphs and single-node-type, multi-relation graphs.

5 Discussion

5.1 Ablation Study: Contribution of AMPSS

To evaluate AMPSS, we compared the full H2S-NN on DBLP node classification with variants using no path-supported 2-cells, predefined meta-path schemas, AMPSS with Stage I only, and AMPSS with Stage II without schema conditioning. As shown in Table 4, removing path-supported 2-cells decreased Macro-F1/Micro-F1, confirming that path-supported 2-cells contribute to node classification. Predefined meta-path schemas recovered most of this loss, while the full AMPSS model performed best. These results suggest that automatic meta-path schema selection can improve performance, but schema-level selection alone is insufficient; schema-conditioned weighting helps filter noisy or uninformative path realizations.

Table 4: Ablation study of AMPSS on DBLP node classification.

Configuration	Macro-F1	Micro-F1
No path-supported 2-cells	93.85	93.92
Predefined meta-path schemas	95.75	96.01
AMPSS, Stage I only	95.88	96.02
AMPSS, Stage II without schema conditioning	95.91	96.08
AMPSS	**96.01**	**96.31**

5.2 Ablation Study: Contribution of SheafCovPool

To evaluate SheafCovPool, we compared the full H2S-NN on the ENZYMES graph classification task using sum, mean, and max pooling, and SheafPool [3]. As shown in Table 5, SheafPool improved Macro-F1/Micro-F1 to 64.05/64.96, indicating that, under this protocol, a gauge-invariant readout retained information not captured by the three plain aggregation baselines. SheafCovPool performed best. Unlike SheafPool, it pools both node and edge Gram matrices and retains attention-weighted means together with across-node and across-edge covariances. These results suggest that gauge invariance is useful but not sufficient, and that retaining second-order statistics of the node and edge cochains is important for graph-level prediction.

Table 5: Ablation study of graph-level readouts on ENZYMES graph classification.

Readout	Macro-F1	Micro-F1
Sum + MLP	61.25	61.79
Mean + MLP	61.42	62.09
Max + MLP	61.38	61.62
SheafPool	64.05	64.96
SheafCovPool	**64.52**	**65.10**

6 Conclusion

To address heterogeneous graph learning under heterophily, we proposed the Heterogeneous Heterophilic Sheaf Neural Network (H2S-NN), a graph neural network for supervised learning tasks. The model preserves concrete meta-path instances as path-supported 2-cells instead of reducing them to endpoint-level relations. It incorporates the ordered boundaries of these path-supported structures into the propagation operator through learned restriction maps and sheaf-Hodge diffusion, allowing information to flow through the node and edge cochains while retaining the relational context of each path. Across the evaluated benchmarks, H2S-NN achieved the strongest performance among the compared methods on node classification and link prediction, and on three of the four graph-classification datasets. Overall, H2S-NN demonstrates that combining the preservation of meta-path and graph structures with sheaf-based propagation provides an effective and generalizable framework for heterogeneous heterophilic graph learning.

Table 1: Performance on node classification benchmarks measured by Macro-F1 and Micro-F1. Best values are bold. The numbers in parentheses after the dataset names indicate the meta-path-based label heterophily (MLH) defined in Equation (5). HetSheaf values are quoted from the original paper [3] because its code is unavailable.

		DBLP (0.1277)		ACM (0.2639)		IMDB (0.7634)	
Group	**Model**	Macro-F1	Micro-F1	Macro-F1	Micro-F1	Macro-F1	Micro-F1
Hom./hom.	GCN	90.84 ± 0.32	91.47 ± 0.34	92.17 ± 0.24	92.12 ± 0.23	57.88 ± 1.18	64.82 ± 0.64
	GAT	93.83 ± 0.27	93.39 ± 0.30	92.26 ± 0.94	92.19 ± 0.93	58.94 ± 1.35	64.86 ± 0.43
Hom./het.	ChebNetII	92.05 ± 0.53	92.97 ± 0.48	92.45 ± 0.37	92.33 ± 0.38	62.54 ± 1.29	68.33 ± 0.92
	NSD	92.96 ± 0.21	93.42 ± 0.37	92.33 ± 0.78	92.05 ± 0.69	63.07 ± 1.05	65.45 ± 0.65
	CSNN	93.73 ± 0.95	92.12 ± 0.19	93.13 ± 0.94	92.81 ± 0.83	62.88 ± 0.62	66.43 ± 0.11
Het./hom.	HAN	91.67 ± 0.49	92.05 ± 0.62	90.89 ± 0.43	90.79 ± 0.43	57.74 ± 0.96	64.63 ± 0.58
	SeHGNN	95.06 ± 0.17	95.42 ± 0.17	94.05 ± 0.35	93.98 ± 0.36	67.11 ± 0.25	69.17 ± 0.43
	HINormer	94.94 ± 0.21	94.57 ± 0.23	91.60 ± 0.47	91.52 ± 0.48	67.83 ± 0.34	64.65 ± 0.53
	PSHGCN	95.27 ± 0.13	95.61 ± 0.12	94.35 ± 0.23	94.27 ± 0.23	72.33 ± 0.57	74.46 ± 0.32
Het./het.	H2G-former	91.99 ± 0.24	92.57 ± 0.24	91.92 ± 0.13	91.95 ± 0.13	65.01 ± 0.63	66.94 ± 0.62
	Hetero^{2}Net	94.03 ± 0.35	94.46 ± 0.37	94.01 ± 0.54	93.91 ± 0.61	65.37 ± 0.48	69.61 ± 0.72
	H2SGNN	95.19 ± 0.11	95.56 ± 0.11	94.47 ± 0.25	94.38 ± 0.26	73.04 ± 0.65	75.46 ± 0.43
	HetSheaf	93.87 ± 0.00	93.77 ± 0.00	95.09 ± 0.00	94.94 ± 0.00	64.65 ± 0.00	67.18 ± 0.00
	LatGRL	92.63 ± 0.61	93.17 ± 0.56	92.54 ± 0.87	92.41 ± 0.89	50.73 ± 0.14	61.34 ± 0.71
Ours	**H2S-NN**	**96.01 ± 0.58**	**96.31 ± 0.40**	**95.91 ± 1.15**	**95.72 ± 0.68**	**75.05 ± 0.22**	**76.68 ± 0.19**

Table 2: Performance on graph classification benchmarks measured by Macro-F1 and Micro-F1. Best values are bold. HetSheaf values are quoted from the original paper [3] because its code is unavailable.

		MUTAG		PROTEINS		ENZYMES		NCI1	
Group	**Model**	Macro-F1	Micro-F1	Macro-F1	Micro-F1	Macro-F1	Micro-F1	Macro-F1	Micro-F1
Hom./hom.	GCN	89.58 ± 0.63	90.00 ± 0.38	75.92 ± 0.55	79.83 ± 0.49	12.75 ± 0.01	28.33 ± 0.03	64.10 ± 0.29	64.92 ± 0.38
	GAT	89.58 ± 0.06	90.00 ± 0.96	75.27 ± 0.59	78.38 ± 0.71	14.03 ± 0.05	16.55 ± 0.12	64.52 ± 0.27	65.26 ± 0.69
Hom./het.	NSD	99.47 ± 0.53	99.69 ± 0.31	73.60 ± 0.22	75.89 ± 0.49	33.65 ± 0.37	35.00 ± 0.21	72.30 ± 0.31	72.30 ± 0.10
	CSNN	99.21 ± 0.79	99.40 ± 0.60	76.64 ± 0.28	77.92 ± 0.07	35.15 ± 0.07	37.22 ± 0.13	74.11 ± 0.75	74.30 ± 0.80
Het./hom.	HAN	82.91 ± 0.61	85.00 ± 0.19	79.19 ± 0.82	82.22 ± 0.50	46.24 ± 0.06	45.02 ± 0.06	77.60 ± 0.43	77.50 ± 0.29
	PSHGCN	94.88 ± 0.10	95.00 ± 0.42	70.61 ± 0.13	72.32 ± 0.74	60.18 ± 0.24	58.33 ± 0.38	74.17 ± 0.34	74.96 ± 0.37
Het./het.	H2G-former	84.65 ± 0.71	85.00 ± 0.64	77.64 ± 0.83	81.25 ± 0.65	58.27 ± 0.21	58.33 ± 0.64	79.29 ± 0.61	79.32 ± 0.18
	Hetero^{2}Net	89.58 ± 0.16	90.00 ± 0.62	74.66 ± 0.69	78.57 ± 0.43	61.53 ± 0.21	61.36 ± 0.49	63.67 ± 0.66	65.45 ± 0.38
	H2SGNN	94.18 ± 0.09	94.55 ± 0.35	71.61 ± 0.37	73.28 ± 0.14	62.18 ± 0.44	61.31 ± 0.45	74.17 ± 0.06	74.96 ± 0.44
	HetSheaf	**100.00 ± 0.00**	**100.00 ± 0.00**	80.56 ± 0.00	82.14 ± 0.00	62.35 ± 0.00	63.33 ± 0.00	79.46 ± 0.00	79.46 ± 0.00
Ours	**H2S-NN**	99.65 ± 0.35	99.72 ± 0.28	**84.44 ± 0.21**	**86.23 ± 0.15**	**64.52 ± 0.56**	**65.10 ± 0.37**	**81.22 ± 0.06**	**81.52 ± 0.20**

Table 3: Performance on link prediction benchmarks measured by AUPR and ROC-AUC. Best values are bold. HetSheaf values are quoted from the original paper [3] because its code is unavailable.

		LastFM		PubMed		Amazon		YouTube	
Group	**Model**	AUPR	ROC-AUC	AUPR	ROC-AUC	AUPR	ROC-AUC	AUPR	ROC-AUC
Hom./hom.	GCN	97.06 ± 0.48	96.96 ± 0.41	96.05 ± 0.52	92.93 ± 0.36	92.40 ± 0.61	89.66 ± 0.73	89.30 ± 0.39	83.26 ± 0.52
	GAT	63.06 ± 0.27	51.14 ± 0.19	96.07 ± 0.43	93.27 ± 0.68	88.62 ± 0.34	86.10 ± 0.45	66.76 ± 0.31	58.40 ± 0.22
Hom./het.	NSD	97.16 ± 0.71	96.58 ± 0.65	96.41 ± 0.58	93.65 ± 0.74	92.63 ± 0.49	90.04 ± 0.38	89.32 ± 0.43	83.36 ± 0.57
	CSNN	97.99 ± 0.42	97.75 ± 0.39	96.14 ± 0.30	92.87 ± 0.29	92.10 ± 0.53	89.62 ± 0.41	88.64 ± 0.34	82.97 ± 0.48
Het./hom.	HAN	88.26 ± 0.37	83.32 ± 0.46	95.78 ± 0.49	93.62 ± 0.62	95.24 ± 0.56	93.64 ± 0.60	92.46 ± 0.51	90.10 ± 0.42
	PSHGCN	88.02 ± 0.44	80.55 ± 0.50	97.58 ± 0.35	94.32 ± 0.21	94.28 ± 0.46	92.34 ± 0.26	93.15 ± 0.63	90.66 ± 0.58
Het./het.	H2G-former	97.89 ± 0.51	97.93 ± 0.45	95.03 ± 0.77	93.15 ± 0.36	93.37 ± 0.40	91.42 ± 0.59	91.68 ± 0.65	88.33 ± 0.38
	Hetero^{2}Net	97.58 ± 0.29	97.45 ± 0.61	94.66 ± 0.53	92.57 ± 0.47	94.71 ± 0.54	92.37 ± 0.52	90.67 ± 0.70	86.45 ± 0.41
	H2SGNN	97.88 ± 0.36	97.22 ± 0.68	95.22 ± 0.42	92.32 ± 0.55	93.18 ± 0.61	92.31 ± 0.50	90.17 ± 0.46	88.96 ± 0.64
	HetSheaf	98.51 ± 0.00	98.00 ± 0.00	96.36 ± 0.00	93.00 ± 0.00	93.76 ± 0.00	91.00 ± 0.00	91.27 ± 0.00	88.00 ± 0.00
Ours	**H2S-NN**	**98.91 ± 0.37**	**98.72 ± 0.35**	**97.63 ± 0.41**	**96.48 ± 0.54**	**98.53 ± 0.56**	**98.33 ± 0.44**	**93.95 ± 0.47**	**91.81 ± 0.33**

References

[1] Federico Barbero, Cristian Bodnar, Haitz Sáez de Ocáriz Borde, Michael Bronstein, Petar Veličković, and Pietro Liò. 2022. Sheaf Neural Networks with Connection Laplacians. In *Proceedings of Topological, Algebraic, and Geometric Learning Workshops 2022 (Proceedings of Machine Learning Research, Vol. 196)*, Alexander Cloninger, Timothy Doster, Tegan Emerson, Manohar Kaul, Ira Ktena, Henry Kvinge, Nina Miolane, Bastian Rieck, Sarah Tymochko, and Guy Wolf (Eds.). PMLR, 28–36. https://proceedings.mlr.press/v196/barbero22a.html

[2] Cristian Bodnar, Francesco Di Giovanni, Benjamin Chamberlain, Pietro Lio, and Michael Bronstein. 2022. Neural sheaf diffusion: A topological perspective on heterophily and oversmoothing in gnns. *Advances in Neural Information Processing Systems* 35 (2022), 18527–18541.

[3] Luke Braithwaite, Alessio Borgi, Gabriele Onorato, Kristjan Tarantelli, Francesco Restuccia, Fabrizio Silvestri, and Pietro Liò. 2024. Heterogeneous sheaf neural networks. *arXiv preprint arXiv:2409.08036* (2024).

[4] Chen Cai and Yusu Wang. 2020. A note on over-smoothing for graph neural networks. *arXiv preprint arXiv:2006.13318* (2020).

[5] Qin Chen and Guojie Song. 2025. Adaptive heterogeneous graph neural networks: Bridging heterophily and heterogeneity. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 312–321.

[6] Federico Errica, Marco Podda, Davide Bacciu, and Alessio Micheli. 2020. A fair comparison of graph neural networks for graph classification. In *Proceedings of the 8th International Conference on Learning Representations (ICLR)*.

[7] Matthias Fey and Jan E. Lenssen. 2019. Fast Graph Representation Learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*.

[8] Matthias Fey, Jinu Sunil, Akihiro Nitta, Rishi Puri, Manan Shah, Blaž Stojanovič, Ramona Bendias, Alexandria Barghi, Vid Kocijan, Zecheng Zhang, Xinwei He, Jan E. Lenssen, and Jure Leskovec. 2025. PyG 2.0: Scalable Learning on Real World Graphs. In *Temporal Graph Learning Workshop at KDD*.

[9] Stefano Fiorini, Hakan Aktas, Iulia Duta, Stefano Coniglio, Pietro Morerio, Alessio Del Bue, and Pietro Liò. 2025. Sheaves Reloaded: A Directional Awakening. *arXiv preprint arXiv:2506.02842* (2025).

[10] Karish Grover, Haiyang Yu, Qi Zhu, Han Xie, Vassilis Ioannidis, Christos Faloutsos, et al. 2025. Spectro-riemannian graph neural networks. In *International Conference on Learning Representations*, Vol. 2025. 57833–57859.

[11] Jiayan Guo, Lun Du, Wendong Bi, Qiang Fu, Xiaojun Ma, Xu Chen, Shi Han, Dongmei Zhang, and Yan Zhang. 2023. Homophily-oriented heterogeneous graph rewiring. In *Proceedings of the ACM web conference 2023*. 511–522.

[12] Jakob Hansen and Thomas Gebhart. 2020. Sheaf neural networks. *arXiv preprint arXiv:2012.06333* (2020).

[13] Jakob Hansen and Robert Ghrist. 2019. Toward a spectral theory of cellular sheaves: J. Hansen, R. Ghrist. *Journal of Applied and Computational Topology* 3, 4 (2019), 315–358.

[14] Mingguo He, Zhewei Wei, Shikun Feng, Zhengjie Huang, Weibin Li, Yu Sun, and Dianhai Yu. 2024. Spectral heterogeneous graph convolutions via positive noncommutative polynomials. In *Proceedings of the ACM Web Conference 2024*. 685–696.

[15] Mingguo He, Zhewei Wei, and Ji-Rong Wen. 2022. Convolutional Neural Networks on Graphs with Chebyshev Approximation, Revisited. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[16] Ferran Hernandez Caralt, Guillermo Bernárdez Gil, Iulia Duta, Pietro Liò, and Eduard Alarcón Cot. 2024. Joint Diffusion Processes as an Inductive Bias in Sheaf Neural Networks. In *Proceedings of the Geometry-grounded Representation Learning and Generative Modeling Workshop (GRaM) (Proceedings of Machine Learning Research, Vol. 251)*. PMLR, 249–263.

[17] Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. 2020. Heterogeneous graph transformer. In *Proceedings of the web conference 2020*. 2704–2710.

[18] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations (ICLR)*.

[19] Jintang Li, Zheng Wei, Yuchang Zhu, Ruofan Wu, Huizhe Zhang, Liang Chen, and Zibin Zheng. 2025. Heterophily-Aware Representation Learning on Heterogeneous Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 47, 9 (2025), 7852–7866. doi:10.1109/TPAMI.2025.3573615

[20] Junhong Lin, Xiaojie Guo, Shuaicheng Zhang, Yada Zhu, and Julian Shun. 2025. When Heterophily Meets Heterogeneity: Challenges and a New Large-Scale Graph Benchmark. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 5607–5618.

[21] Xiaozhong Liu, Yingying Yu, Chun Guo, and Yizhou Sun. 2014. Meta-path-based ranking with pseudo relevance feedback on heterogeneous graph for citation recommendation. In *Proceedings of the 23rd acm international conference on conference on information and knowledge management*. 121–130.

[22] Kangkang Lu, Yanhua Yu, Ruopei Guo, Nan Cheng, Zhiyong Huang, Yunshan Ma, Meiyu Liang, Yuling Wang, Xiting Qin, Yimeng Ren, et al. 2026. Addressing graph heterogeneity and heterophily from a spectral perspective. *Neurocomputing* (2026), 133500.

[23] Qingsong Lv, Ming Ding, Qiang Liu, Yuxiang Chen, Wenzheng Feng, Siming He, Chang Zhou, Jianguo Jiang, Yuxiao Dong, and Jie Tang. 2021. Are we really making much progress? Revisiting, benchmarking, and refining heterogeneous graph neural networks. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*. arXiv:2112.14936

[24] Qiheng Mao, Zemin Liu, Chenghao Liu, and Jianling Sun. 2023. Hinormer: Representation learning on heterogeneous information networks with graph transformer. In *Proceedings of the ACM web conference 2023*. 599–610.

[25] Alexander Mathiasen, Frederik Hvilshøj, Jakob Rødsgaard Jørgensen, Anshul Nasery, and Davide Mottin. 2020. What if neural networks had svds? *Advances in Neural Information Processing Systems* 33 (2020), 18411–18420.

[26] Christopher Morris, Nils M Kriege, Franka Bause, Kristian Kersting, Petra Mutzel, and Marion Neumann. 2020. Tudataset: A collection of benchmark datasets for learning with graphs. *arXiv preprint arXiv:2007.08663* (2020).

[27] Emanuele Mule, Stefano Fiorini, Antonio Purificato, Federico Siciliano, Stefano Coniglio, and Fabrizio Silvestri. 2025. Directional Sheaf Hypergraph Networks: Unifying Learning on Directed and Undirected Hypergraphs. arXiv:2510.04727 [cs.LG] https://arxiv.org/abs/2510.04727

[28] Antonio Purificato, Giulia Cassarà, Federico Siciliano, Pietro Liò, and Fabrizio Silvestri. 2025. Sheaf4rec: Sheaf neural networks for graph-based recommender systems. *ACM Transactions on Recommender Systems* 4, 2 (2025), 1–26.

[29] André Ribeiro, Ana Luiza Tenorio, Juan Belieni, Amauri H Souza, and Diego Mesquita. 2026. Cooperative Sheaf Neural Networks. In *The Fourteenth International Conference on Learning Representations*. https://openreview.net/forum?id=AHpexliCTM

[30] Zhixiang Shen and Zhao Kang. 2025. When heterophily meets heterogeneous graphs: Latent graphs guided unsupervised representation learning. *IEEE Transactions on Neural Networks and Learning Systems* (2025).

[31] Deepti Singh and Ankita Verma. 2025. An overview of heterogeneous social network analysis. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* 15, 2 (2025), e70028.

[32] Yizhou Sun and Jiawei Han. 2013. Mining heterogeneous information networks: a structural analysis approach. *ACM SIGKDD explorations newsletter* 14, 2 (2013), 20–28.

[33] Yizhou Sun, Jiawei Han, Xifeng Yan, Philip S Yu, and Tianyi Wu. 2011. Pathsim: Meta path-based top-k similarity search in heterogeneous information networks. *Proceedings of the VLDB Endowment* 4, 11 (2011), 992–1003.

[34] Zhen Tao, Ziyue Qiao, Chaoqi Chen, Zhengyi Yang, Lun Du, and Qingqiang Sun. 2025. On the Cross-type Homophily of Heterogeneous Graphs: Understanding and Unleashing. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 2842–2852.

[35] Laurens Van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-SNE. *Journal of machine learning research* 9, 11 (2008).

[36] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations (ICLR)*.

[37] Haosen Wang, Chenglong Shi, Can Xu, Surong Yan, Rong Xie, and Pan Tang. 2025. Enhancing Homophily in Heterogeneous Graph Contrastive Learning via Connection Strength and Multi-view Self-Expression. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Padua, Italy) *(SIGIR '25)*. Association for Computing Machinery, New York, NY, USA, 709–718. doi:10.1145/3726302.3729968

[38] Xiao Wang, Houye Ji, Chuan Shi, Bai Wang, Yanfang Ye, Peng Cui, and Philip S Yu. 2019. Heterogeneous graph attention network. In *The world wide web conference*. 2022–2032.

[39] Xiaocheng Yang, Mingyu Yan, Shirui Pan, Xiaochun Ye, and Dongrui Fan. 2023. Simple and Efficient Heterogeneous Graph Neural Network. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

[40] Juwei Yue, Haikuo Li, Jiawei Sheng, Yihan Guo, Xinghua Zhang, Chuan Zhou, Tingwen Liu, and Li Guo. 2025. Graph wave networks. In *Proceedings of the ACM on Web Conference 2025*. 1365–1379.

[41] Chuxu Zhang, Dongjin Song, Chao Huang, Ananthram Swami, and Nitesh V Chawla. 2019. Heterogeneous graph neural network. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 793–803.

[42] Muhan Zhang, Zhicheng Cui, Marion Neumann, and Yixin Chen. 2018. An end-to-end deep learning architecture for graph classification. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 32.

[43] Jiong Zhu, Yujun Yan, Lingxiao Zhao, Mark Heimann, Leman Akoglu, and Danai Koutra. 2020. Beyond homophily in graph neural networks: Current limitations and effective designs. *Advances in neural information processing systems* 33 (2020), 7793–7804.